

```
"""
```

```
StegVerse Full Pipeline Runner v1.0
```

```
Invoked as a single declared task by core-lite-intake.yml.
```

```
Runs: triad-sandbox → triad-replay → stegclaw-session → site-data-pipeline  
in dependency order with bounded retry and fail-closed on errors.
```

```
NO new workflow files. This is a tool, not a workflow.
```

```
"""
```

```
import json
```

```
import subprocess
```

```
import sys
```

```
import time
```

```
from datetime import datetime, timezone
```

```
from pathlib import Path
```

```
SCHEMA = "stegverse_full_pipeline_report.v1"
```

```
def now() -> str:
```

```
    return datetime.now(timezone.utc).isoformat()
```

```
def run_step(name: str, command: list, expected_outputs: list, retries: int = 1)
```

```
    print(f"\n[{name}] Starting...")
```

```
    for attempt in range(1, retries + 2):
```

```
        try:
```

```
            result = subprocess.run(
```

```
                command,
```

```
                capture_output=True, text=True, timeout=300
```

```
            )
```

```
            if result.returncode == 0:
```

```
                missing = [p for p in expected_outputs if not Path(p).exists()]
```

```
                if missing:
```

```
                    print(f"[{name}] FAIL - missing outputs: {missing}")
```

```
                    if attempt <= retries:
```

```
                        time.sleep(5)
```

```
                        continue
```

```
                    return {"step": name, "status": "FAIL_CLOSED",
```

```
                            "reason": f"Missing outputs: {missing}"}
```

```
                print(f"[{name}] ALLOW")
```

```
                return {"step": name, "status": "ALLOW",
```

```
                        "outputs": expected_outputs, "stdout": result.stdout[-500
```

```
            else:
```

```
                print(f"[{name}] returncode={result.returncode}")
```

```
                print(result.stderr[-500:] if result.stderr else "")
```

```
                if attempt <= retries:
```

```

        print(f"[{name}] Retrying ({attempt}/{retries})...")
        time.sleep(5)
        continue
    return {"step": name, "status": "FAIL_CLOSED",
            "reason": f"Non-zero exit: {result.returncode}",
            "stderr": result.stderr[-500:]}
except subprocess.TimeoutExpired:
    return {"step": name, "status": "FAIL_CLOSED", "reason": "Timeout"}
except FileNotFoundError as e:
    return {"step": name, "status": "FAIL_CLOSED", "reason": f"Command not found: {e}"}
return {"step": name, "status": "FAIL_CLOSED", "reason": "Max retries exceeded"}

```

```

PIPELINE_STEPS = [
    {
        "name": "triad-sandbox",
        "command": [
            "python", "triad/sandbox_adapter.py",
            "--vectors", "triad/candidate_vectors",
            "--receipts", "triad/brain_reports/triad_receipts.jsonl",
            "--report", "triad/brain_reports/triad_sandbox_report.json",
            "--summary", "triad/brain_reports/triad_sandbox_summary.md",
        ],
        "expected_outputs": ["triad/brain_reports/triad_receipts.jsonl"],
        "retries": 1,
        "required": True,
    },
    {
        "name": "triad-replay",
        "command": [
            "python", "triad/receipt_replay.py",
            "--receipts", "triad/brain_reports/triad_receipts.jsonl",
            "--vectors", "triad/candidate_vectors",
            "--report", "triad/brain_reports/triad_replay_report.json",
            "--summary", "triad/brain_reports/triad_replay_summary.md",
        ],
        "expected_outputs": ["triad/brain_reports/triad_replay_report.json"],
        "retries": 0,
        "required": True,
        "depends_on": ["triad-sandbox"],
    },
    {
        "name": "stegclaw-session",
        "command": ["python", "stegclaw.py"],
        "env_required": [
            "STEGCLAW_USER_ID",
            "STEGCLAW_SUBJECT_ID",
        ],
    },
]

```

```

    ],
    "expected_outputs": [
        "brain_reports/next_action.json",
        "brain_reports/stegclaw_run_report.json",
    ],
    "retries": 1,
    "required": False,    # non-critical - pipeline continues if vault not yet
    "depends_on": ["triad-replay"],
},
{
    "name": "site-data-pipeline",
    "command": [
        "python", "tools/site_data_pipeline.py",
        "tools/site_pipeline_config.json",
        "site_data",
    ],
    "expected_outputs": [
        "site_data/transition_release_index.json",
        "site_data/transition_dev_status.json",
    ],
    "retries": 0,
    "required": True,
    "depends_on": ["triad-sandbox"],
},
]

```

```

def run_pipeline() -> dict:
    import os
    results    = []
    completed  = set()
    blocked    = set()

    print(f"=== Full Pipeline ===   {now()}")

    for step in PIPELINE_STEPS:
        name = step["name"]

        # Dependency check
        deps = step.get("depends_on", [])
        failed_deps = [d for d in deps if d in blocked]
        if failed_deps:
            print(f"\n[{name}] SKIPPED - blocked deps: {failed_deps}")
            blocked.add(name)
            results.append({"step": name, "status": "SKIPPED",
                           "reason": f"Blocked by: {failed_deps}"})
            continue

```

```

# Env check
env_required = step.get("env_required", [])
missing_env = [e for e in env_required if not os.environ.get(e)]
if missing_env:
    print(f"\n[{name}] SKIPPED - missing env: {missing_env}")
    if step.get("required"):
        blocked.add(name)
    results.append({"step": name, "status": "SKIPPED",
                    "reason": f"Missing env vars: {missing_env}"})
    continue

result = run_step(name, step["command"], step["expected_outputs"], step.g
results.append(result)

if result["status"] == "ALLOW":
    completed.add(name)
else:
    if step.get("required"):
        blocked.add(name)
    print(f"[{name}] Required step failed - downstream steps blocked.

# Summary
allow_count = sum(1 for r in results if r["status"] == "ALLOW")
fail_count = sum(1 for r in results if r["status"] == "FAIL_CLOSED")
skip_count = sum(1 for r in results if r["status"] == "SKIPPED")

report = {
    "schema": SCHEMA,
    "generated_at": now(),
    "total_steps": len(results),
    "allow": allow_count,
    "fail_closed": fail_count,
    "skipped": skip_count,
    "steps": results,
    "overall": "ALLOW" if fail_count == 0 else "FAIL_CLOSED",
}

Path("brain_reports").mkdir(exist_ok=True)
report_path = Path("brain_reports/full_pipeline_report.json")
with open(report_path, "w") as f:
    json.dump(report, f, indent=2)

print(f"\n=== Pipeline complete: {allow_count} ALLOW, {fail_count} FAIL, {ski
print(f"Report: {report_path}")

return report

```

```
if __name__ == "__main__":  
    report = run_pipeline()  
    sys.exit(0 if report["overall"] == "ALLOW" else 1)
```